算子迁移任务报告：

- 任务：用 `refs/tilelang-mlir-ascend/.opencode/agents` 的 OpenCode conductor + subagent 框架，把 `refs/TileOPs` 中 5 个 GPU 算子迁移到 Ascend 910B3 NPUIR。流程固定为 Stage 0（脚手架）→ Stage 1（设计）→ Stage 2（设计评审）→ Stage 3（开发）→ Stage 5（集成验收），Stage 4 性能调优跳过。
- 结论：**5/5 算子迁移完成，框架侧验收全部通过；独立复验全部通过。任务主体完成，存在若干框架/数据层面的遗留问题（不影响功能验收），Stage 4 未做，性能数据仅为基线。**

---

1. 验收结论

验收项	判据	结果
迁移范围	5 个指定算子全部产出 NPUIR kernel 并集成进 TileOPs	☑ 5/5
框架状态	每算子顶层 `.stage_state.json` `phase=DONE`、`.migration_state.json` `integration=pass`	☑ 5/5
集成报告	`reports/tileops/<run_id>/run.json` `status=passed`，correctness + benchmark 退出码均为 0	☑ 5/5
设计/评审留痕	每算子有 DESIGN.md、REVIEW.md、RETROSPECTIVE.md、verify_equiv.py、函数级 timeline	☑ 5/5
独立复验	不经框架 CLI，直接复跑 `tests/ops/*.py`	☑ 5/5（见第 3 节）

判定：功能验收 PASS；工程交付 CONDITIONAL PASS（代码未入库、Stage 4 未做、Erf 报告数据口径混入 Expm1）。

---

2. 五个算子的最终结果

算子	slug	正确性（算子自身用例）	benchmark（算子自身）	最大绝对误差	墙钟	返工
LeakyReluFwdOp	`leaky_relu`	14/14	9 条 / 6 shape	0	2 h 37 min	gate1 失败 1 次 + 评审打回 1 次
EluFwdOp	`elu_fwd`	14/14	9 条 / 6 shape	0	1 h 49 min	Stage 1 runtime 失败 1 次
Expm1FwdOp	`expm1`	8/8	10 条 / 7 shape	0	2 h 36 min	评审打回 1 次
ErfFwdOp	`erf_fwd`	8/8（共享文件 16/16）	10 条 / 7 shape（共享文件 20 条）	fp32 6.74e-7 / fp16 1.22e-4 / bf16 7.63e-6	1 h 58 min	无，一次通过
BitwiseNotFwdOp	`bitwise_not`	9/9	8 条 / 5 shape	0（位级精确）	1 h 20 min	Stage 1 runtime 失败 1 次

说明：

- `expm1` 与 `erf` 共用 `tests/ops/test_unary_math.py`。Expm1 集成时该文件只有 8 条 expm1 用例；Erf 集成时追加了 8 条 erf 用例，所以 Erf 的 report 显示 16/16，其中只有 8 条属于 Erf。
- 同理 `benchmarks/ops/bench_unary_elementwise.py` 同时参数化 Expm1 和 Erf，Erf 的 report 因此有 20 条 bench 记录，前 10 条 operator 字段是 `Expm1FwdOp`。引用 Erf 性能时只能取 `ErfFwdOp` 的 10 条。
- 框架 `report.md` 对 Erf 写的分母是 10/10，`run.json.summary` 是 16，RETROSPECTIVE 是 8/8；**Erf 自身用例的真实数量是 8**，这是框架统计口径不一致，不是测试失败。

实现路线（各算子）

算子	NPU 实现要点
LeakyRelu	`vselect` 比较 + 乘法；fp32 分支原地写回，避免 stale buffer
Elu	`vselect` + `vexp`；fp16/bf16 中转 fp32 计算，fp32 原生路径
Expm1	`vexp` + `vsub(., 1.0)` 链；fp32 原生、fp16 中转、bf16 走 fp32；persistent 分核
Erf	Abramowitz & Stegun 7.1.26 近似（22 条 v-prefix op 链）；**禁用** `T.verf`（实测为 7 阶泰勒，$\|x\| \geq 1$ 失真）
BitwiseNot	整数走原生 `vnot`；uint8 走 int8 bit-view + `vnot`；bool 走 int8 bit-view + `fill(1)` + `vxor`；1D persistent 分核，`block_size=256`

---

3. 独立复验（不采信框架自述）

方法：不经过 `tileops.reporting.cli`，直接在各算子集成目录运行测试文件；环境 `PYTHONPATH=/mnt/workspace/learn/tilelang-ascend:<TileOPs 根>`、`TILELANG_ASCEND_MODE=Developer`。脚本：`notes/experiments/rerun_all_acceptance.sh`；原始日志：`logs/independent/`。

2026-10-29 08:21–08:29 复跑结果（全部 exit 0，合计 **53 passed / 0 failed**，用时 397 s）：

测试文件	覆盖算子	结果	耗时
`tests/ops/test_leaky_relu.py`	LeakyReluFwdOp	14 passed	105.43 s
`tests/ops/test_elu.py`	EluFwdOp	14 passed	96.45 s
`tests/ops/test_unary_math.py`	Expm1FwdOp + ErfFwdOp	16 passed（8 + 8）	100.17 s
`tests/ops/test_bitwise_not.py`	BitwiseNotFwdOp	9 passed	95.09 s

结论：5 个算子的独立复验与框架 report 一致，没有"框架自述通过、实际失败"的情况。日志原文见 `logs/independent/acceptance_*.log`。

---

4. 性能基线（msprof device kernel 时间）

完整 43 条记录见 `benchmarks/summary.csv`；下表只列各算子的代表点。

算子	规模 / dtype	kernel 时延	有效带宽*	框架 Ratio
LeakyRelu	1 M fp32	17.38 μs	482.6 GB/s	13.79%
LeakyRelu	16.8 M fp16	141.46 μs	474.4 GB/s	13.55%
Elu	1 M fp32	18.94 μs	442.9 GB/s	12.65%
Elu	16.8 M fp16	155.90 μs	430.5 GB/s	16.45%
Expm1	1 M fp32	16.46 μs	509.6 GB/s	14.56%
Expm1	256 M fp16	2053.02 μs	523.0 GB/s	27.08%
Erf	1 M fp32	30.10 μs	278.7 GB/s	18.66%
Erf	256 M fp16	4702.47 μs	228.3 GB/s	31.32%
BitwiseNot	1 M bool	29.24 μs	71.7 GB/s	1.02%
BitwiseNot	256 M int32	7220.36 μs	297.4 GB/s	16.20%

* 有效带宽 = `2 × numel × elem_bytes / latency`，按读写各一次计算。这是本文档自己算的口径，不是框架字段。

口径提醒：

- 时延是 **msprof 采集的 device kernel 时间**，不是 host wall time。Day 2 已确认 host 开销 250–450 μs，会淹没这类小算子，因此不能拿端到端时间评估 kernel。
- 框架输出里的 `Bandwidth(TB/s)=1.75` 在所有记录中是常数，是 roofline 模型参数而非实测；`Ratio(%)` 的分母反推约 3.5 TB/s，与 1.75 对不上，**口径未确认，本文只作参考，不作为性能结论。**
- Stage 4 跳过，以上全部是**未调优基线。**Erf 是 22-op 超越函数链，计算受限，有效带宽低是预期结果；BitwiseNot 的 bool/uint8 受小元素字节与固定开销影响，带宽数字不具可比性。

---

5. 过程效率

按 `.task_timeline.jsonl` 统计（含返工）：

阶段	耗时	占比
Stage 0 脚手架	58.0 min	9.4%
Stage 1 设计 + Stage 2 评审（含失败重试）	350.7 min	56.6%
Stage 3 开发（真正写 kernel + 自测）	77.3 min	12.5%
Stage 5 集成 + 全量 report	124.8 min	20.1%
场景路由/收束/间隔	≈9 min	1.5%
合计	**≈620 min（10.3 h）**	100%

关键观察：

- **真正写 kernel 只占 12.5%**，设计与评审（含返工）占 56.6%。瓶颈在设计与机械门禁，不在编码。

- 5 个算子里 4 个发生过至少一次返工；Stage 2 评审对 LeakyRelu 的 2 条阻塞问题（factory 契约、fp32 写回路径）是真实拦截，避免了错误实现进入开发。
- 队列 v2 的"进程消失即用 `--session` 续跑"策略生效：Elu/Expm1/Erf/BitwiseNot 在 8 h 54 min 内串行跑完，无任务丢失。

各算子阶段耗时（min）：

算子	脚手架	设计	评审	开发	集成	失败重试
LeakyRelu	13.0	29.4	16.3	23.9	21.6	31.1 + 20.6
Elu	11.2	8.5	16.8	12.4	21.7	35.1
Expm1	12.9	67.2	13.9	12.5	24.4	24.3
Erf	10.8	39.4	14.6	14.4	36.1	—
BitwiseNot	10.2	0.9	12.7	14.1	21.0	19.9

---

6. 过程中确认的关键事实 / 踩过的坑

1. `T.verf` **不是硬件 erf**：实测是 7 阶泰勒 `(2/√π)(x−x³/3+x⁵/10−x⁷/42)`，|x|=1 误差 4.5e-3、|x|=3 误差 39。Erf 改用 A&S 7.1.26，离线 2 M 点最大绝对误差 5.8e-7。
2. **uint8 的** `out_idx=[1]` **会把输出分配成 float32**：计算本身没错，是 dtype 分配错误。规避：`out_idx=[]` + 调用方 `empty_like`，或转 int8 视图。
3. **原生** `dtype="bool"` **编译通过但执行挂死**：bool 一律用 int8 bit-view；且 `vnot` 对 bool 不成立（`~0 = 0xFF` 仍为真），必须 `fill(ones,1)` + `vxor`。
4. **2D** `T.Kernel` **第二维会被静默串行化**（Day 4 结论，本轮沿用）：一律 1D 扁平 grid。
5. `vbrc` **标量绑定必须写在** `@T.prim_func` **体内**，写在工厂作用域会报 `input vector and output vector must have same rank`。
6. **wrapper 集成必须按两段式调用面适配**：`factory(N, dtype_str, block_size=bs)(bs)(x)`；单段或保留尾部 `, N` 会在 build 期失败。
7. `integrate_kernel.py` **的 wrapper 改写不幂等**：scaffold 预埋的 selection 注释块会残留，出现重复块头/重复 perf_opt 注释（Elu/Erf/BitwiseNot 都出现过）。功能无害，需人工清理；已写进各算子 RETROSPECTIVE。
8. **集成 smoke 只验证 import，不验证调用面**：Expm1 的旧 wrapper 调用 `(x, N)` 能通过 smoke，问题要等到全量 report 才暴露。

---

7. 遗留问题与风险

#	问题	影响	建议
1	Erf report 混入 10 条 Expm1 benchmark 记录；correctness 分母 10/16/8 三种口径	引用数据易错	报告/引用只取 `operator=ErfFwdOp` 的记录；Erf 自身 8 条用例
2	迁移代码（kernels/ops/tests/bench/manifest）未 git commit，全部 untracked	交付依赖快照	评审后按算子或整体提交
3	Stage 4 未做	性能为未调优基线	如需性能结论，单独跑 Stage 4
4	框架 `Ratio` / `Bandwidth` 字段口径未确认	不能作为性能结论	以 msprof 时延与自算有效带宽为准
5	`/tmp` 队列日志因会话重启丢失	队列级过程证据缺失	算子级 timeline/report 完整，可接受；后续把队列日志落到仓库目录
6	LeakyRelu 的 `SESSION_TIMING_ANALYSIS.md` 格式无效（`breakdown[10].duration_s must be a number`）	该算子 report 的 Total Time 为 N/A	用 `.task_timeline.jsonl` 计时
7	`integrate_kernel.py` 选择块改写不幂等、smoke 覆盖不足	集成期返工/人工清理	修框架脚本（已在 RETROSPECTIVE 提改进项）
8	未做跨算子统一回归（只按算子分文件跑）	组合回归风险	可补一次 `pytest tests/ops` 全量

8. 复现方法

```
export CANNBOT_GUI=1
export GPU_REPO=/mnt/workspace/learn/refs/TileOPs
export PYTHONPATH=/mnt/workspace/learn/tilelang-
ascend:/mnt/workspace/learn/refs/tilelang-mlir-ascend/examples/TileOPs
export TILELANG_ASCEND_MODE=Developer

# 独立复验（本报告第 3 节的数据来源；在交付包根目录执行）
bash notes/experiments/rerun_all_acceptance.sh

# 单算子重跑框架（会重新生成 DESIGN/REVIEW/kernel/report）
cd /mnt/workspace/learn/refs/tilelang-mlir-ascend
opencode run --agent tilelang-op-conductor --auto --command migrate-op ErfFwdOp

# 只重跑集成报告
cd examples/TileOPs
python -m tileops.reporting.cli run --op ErfFwdOp --prof-mode msprof
```

前置：`.opencode/agents/*.md` 的 model 已改为 CANNBOT 内置模型名（见 `framework/agents-and-integration.patch`）。

9. 交付包内容

> 本节按 2026-10-29 目录重组后的实际结构更新（原 `day5/` 树见 `迁移报告.md`），改动仅限本节。

```
infra-learning/day4/migration/
  迁移报告.md / 迁移报告.pdf              本报告
  notes/
    README.md / STATUS.md                 复现说明、状态摘要
    worklog.md / learning-report.md       过程记录与学习报告
    logs/                                 队列快照、LeakyRelu 时间线、
run/benchmark/report 拷贝、5 份算子 report
    logs/independent/                     4 份独立复验日志（leaky_relu / elu /
expm1+erf / bitwise_not）
    experiments/                          独立复验脚本（rerun_all_acceptance.sh 等）、
uint8/bool/erf 排障 spike
  artifacts/
    examples/<slug>/                      每算子 DESIGN / REVIEW / RETROSPECTIVE /
kernel / verify_equiv / timeline / state
    integrated/                           集成后的 kernels / ops / tests /
benchmarks / manifest_elementwise.yaml / workloads_elementwise.py
    reports/tileops/<run_id>/             5 个 run_id + latest.json；run.json /
benchmark.json / report.md / report.html / pytest（不含原始 msprof artifact）
  framework/
    agents/                               本轮使用的 8 个 OpenCode agent 定义（另含
README）
    agents-and-integration.patch          框架 agent 改动 + 集成代码改动 diff
    git-status.txt / recent-commits.txt 迁移时的仓库状态与 5 条 evolution 提交
  benchmarks/summary.csv                  43 条性能记录（时长、有效带宽、框架 Ratio）
  logs/independent/                       4 份独立复验日志（与 notes/logs/independent
同源）
```

未打包：`reports/*/artifacts/msprof/` 原始 profiler 目录（约 118 MB），需要时用 `tileops.reporting.cli` 重跑即可再生成；关键时延已抽取到 `benchmark.json` 与 `benchmarks/summary.csv`。